



DataPipe

Guide Officiel de l'Équipe

ETL Visuel · Intelligence Artificielle · Hackathon J.U.I.N 2026

HACKATHON	THÈME	ÉQUIPE	DURÉE
J.U.I.N 2026	Thème 09	5 membres	24 heures

"Donner à n'importe quel analyste, sans une ligne de code, la puissance d'un ingénieur data — en local, en français, en 15 minutes."

Table des matières

01	Le projet DataPipe — Vue d'ensemble
02	Innovation & Importance du projet
03	Cibles & Marché réel
04	Stack technique & Architecture
05	Les 12 nœuds du pipeline — Détail complet
06	Répartition des tâches — 5 membres
07	Collaboration P1 & P2 — Règles sans conflits
08	Charte graphique — Tokens & Composants
09	Jalons sur 24h — Planning
10	Le script de démo parfait

Le projet DataPipe

ETL Visuel pour Pipelines Bancaires · Thème 09

Concept

DataPipe est une interface web nodale permettant de construire des pipelines de traitement de données par glisser-déposer. L'utilisateur assemble des blocs visuels (CSV, JSON, SQL, Filtre, Join, Agrégation...) et les connecte entre eux. La donnée "coule" de gauche à droite, chaque nœud affichant son résultat en temps réel.

Ce que les jurés verront en 3 minutes

- 1 Upload d'un CSV de transactions bancaires (1 247 lignes)
- 2 Ajout d'un nœud Filtre : montant > 50 000 FCFA
- 3 Join avec un JSON clients sur client_id
- 4 Agrégation GROUP BY région + SUM(montant)
- 5 Prompt IA en français → pipeline généré automatiquement
- 6 Bar chart interactif + export CSV du résultat

Innovation & Importance

Pourquoi ce projet dépasse la liste

Est-il innovant ?

Des outils ETL visuels existent déjà : n8n, Airbyte, Apache NiFi, Fivetran. L'idée n'est pas nouvelle. Ce qui est innovant dans DataPipe c'est la combinaison spécifique :

Composant	Existant	DataPipe
Interface nodale	✓ Oui	✓ Oui
Moteur SQL léger local	✗ Non (cloud)	✓ DuckDB
Génération par langage naturel	✗ Rare	✓ Claude API
Sans infrastructure	✗ Compte SaaS	✓ 100% local
Coût	\$200-500/mois	Gratuit

Comparaison avec les autres thèmes

Thème	Innovation tech	Effet démo	Vendabilité	Pertinence locale
09 · DataPipe	★★★★★	★★★★★	★★★★	★★★★
12 · RegTech AfCFTA	★★★	★★★	★★★★★	★★★★★
11 · FinAudit	★★★★	★★★★	★★★★★	★★★★
05 · Corporate Wiki RAG	★★★★	★★★	★★★★★	★★★
06 · TaskFlow	★★★	★★★	★★★	★★★

Cibles & Marché réel

Qui a besoin de DataPipe dans la vraie vie

Le problème réel

Un contrôleur de gestion dans une banque ivoirienne reçoit chaque mois 4 fichiers CSV de différentes agences. Il passe 2 jours à copier-coller dans Excel pour les fusionner, filtrer, et produire un rapport. Il ne sait pas coder. DataPipe lui permet de faire ça en 15 minutes, visuellement, sans écrire une seule ligne de code.

Cibles par segment

Segment	Profil	Usage concret	Urgence
Cible 1 Primaire	Contrôleurs de gestion Chargés de reporting Auditeurs	Consolidation de fichiers CSV/Excel hétérogènes chaque semaine	HAUTE
Cible 2 Secondaire	Startups africaines PME en croissance Cabinets comptables	Pas les moyens d'un Fivetran à 500\$/mois — besoin d'outil accessible	HAUTE
Cible 3 Tertiaire	DSI moyennes entreprises Équipes IT sans data engineer	Prototyper des flux de données sans développeur dédié	MOYENNE
Cible 4 Académique	Chercheurs Étudiants data/finance Laboratoires	Croiser des bases de données hétérogènes pour la recherche	FAIBLE

Pourquoi c'est pertinent en Afrique

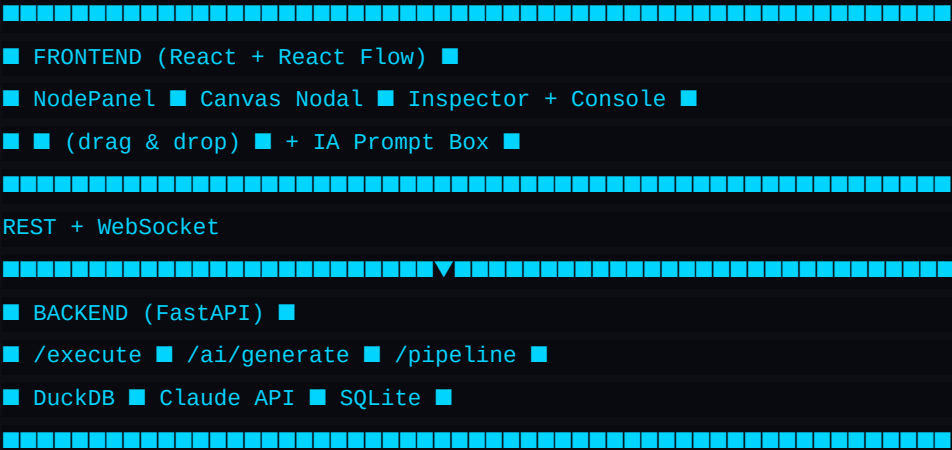
Fragmentation des systèmes	Beaucoup d'entreprises africaines utilisent des logiciels différents par département — souvent des exports CSV car les APIs n'existent pas. La donnée est là mais éparpillée.
Pénurie d'ingénieurs data	Les profils Data Engineer sont rares et chers. Un outil no-code de pipeline comble ce gap immédiatement sans recruter.
Coût des solutions existantes	Fivetran, Airbyte Cloud, Talend coûtent des centaines de dollars par mois. DataPipe tourne en local, coût infrastructure zéro.

Stack Technique & Architecture

Les choix technologiques et pourquoi

Couche	Technologie	Raison du choix
Frontend	React + React Flow	Interface nodale native, parfaite pour les canvas drag & drop
Styling	Tailwind CSS + shadcn/ui	Beau rapidement, tokens cohérents, pas de temps perdu sur le CSS
Backend	FastAPI (Python)	Rapide à écrire, typage fort avec Pydantic, excellent pour la data
Data Engine	DuckDB	SQL sur CSV/JSON en mémoire, ultra rapide, zéro configuration
IA	Claude API (Anthropic)	Génération de pipelines et SQL depuis texte naturel en français
Temps réel	WebSockets	Animation des nœuds pendant l'exécution — effet visuel fort
Persistence	SQLite	Sauvegarde des pipelines créés, léger, zéro infrastructure
Charts	Recharts	Graphiques dans la console output, natif React

Architecture globale



Les 12 Nœuds — Détail complet

Description, config et aperçu de chaque bloc

Nœud	Catégorie	Entrée	Sortie	Config clé
01 · CSV Import	SOURCE	Fichier .csv	DataFrame	Séparateur, encoding, header
02 · JSON Loader	SOURCE	Fichier .json ou URL API	DataFrame aplati	Chemin clé racine
03 · SQL Query	SOURCE	Connexion DB	Résultat requête	Éditeur SQL intégré
04 · Filtre	TRANSFORM	DataFrame	DataFrame filtré	Colonne + condition (>, <, =, ...)
05 · Join	TRANSFORM	2 DataFrames	DataFrame fusionné	Clé jointure + type (LEFT/INNER)
06 · Agrégation	TRANSFORM	DataFrame	DataFrame agrégé	GROUP BY + SUM/COUNT/AVG/MAX
07 · Rename Cols	TRANSFORM	DataFrame	Colonnes modifiées	Renommer, supprimer, réordonner
08 · Nettoyage	TRANSFORM	DataFrame	DataFrame propre	Doublons, nulls, trim strings
09 · IA Transform	IA ■	DataFrame + prompt	DataFrame + code	Prompt libre en français
10 · Table Preview	OUTPUT	DataFrame final	Tableau interactif	Pagination, tri par colonne
11 · Chart	OUTPUT	DataFrame agrégé	Graphique Recharts	Axe X, Axe Y, type (Bar/Line/Pie)
12 · Export	OUTPUT	DataFrame final	Fichier .csv/.json	Format + encoding

Note IA : Le nœud 09 "IA Transform" est le nœud bonus. Il génère automatiquement du SQL DuckDB depuis un prompt en français. C'est la feature qui vaut les +3 points de bonus.

Répartition des tâches — 5 membres

Qui fait quoi, quand et pourquoi

P1 — Lead Frontend · Canvas

Stack : React Flow, Tailwind

Horaire	Tâche
H+0 → H+2	Setup React + React Flow + Tailwind. Layout global 3 colonnes. Canvas vide avec grille de fond.
H+2 → H+5	Drag & drop des nœuds depuis le panel vers le canvas. Connexions entre nœuds (flèches colorées).
H+5 → H+10	Design visuel des 12 nœuds. Sélection d'un nœud → Inspector s'ouvre. Bouton Exécuter navbar.
H+10 → H+16	Animation d'exécution nœud par nœud (allumage vert progressif via WebSocket). Badges de résultat.
H+16 → H+24	Polish visuel, dark mode, logo. Gestion erreurs visuelles. Tests et bugfix.

P2 — Frontend Inspector · Console

Stack : React, shadcn/ui, Recharts

Horaire	Tâche
H+0 → H+2	Setup panneau Inspector (droite). Composant réutilisable par type de nœud.
H+2 → H+6	Formulaires de config pour chaque nœud. Persistance config dans le state React.
H+6 → H+10	Console / Data Preview en bas. Table scrollable. Métadonnées (nb lignes, taille, colonnes).
H+10 → H+14	Composant Chart avec Recharts. Bar, Line, Pie — sélectionnable.
H+14 → H+20	Panel de gauche (liste nœuds). Organisé par catégorie. Drag depuis panel vers canvas.
H+20 → H+24	Polish, responsive, bugfix.

P3 — Backend Core · Data Engine

Stack : FastAPI, DuckDB, Python, SQLite

Horaire	Tâche
H+0 → H+2	Setup FastAPI + structure routes. Modèles Pydantic. Connexion DuckDB en mémoire.
H+2 → H+6	Route POST /execute. Reçoit le pipeline JSON. Exécution topologique. Résultats par nœud.
H+6 → H+10	Implémentation des transformations : Filtre, Join, Agrégation, Rename, Nettoyage via DuckDB.
H+10 → H+14	Upload CSV/JSON. Route POST /upload. Détection auto du schéma.

H+14 → H+18	WebSocket pour exécution temps réel. Event par nœud : {nodeId, status, rowCount, preview}.
H+18 → H+24	Save/Load pipeline dans SQLite. Tests + gestion erreurs.

P4 — IA Integration

Stack : Python, Claude API, FastAPI

Horaire	Tâche
H+0 → H+3	Setup Claude API. Comprendre le format JSON des pipelines attendus par React Flow.
H+3 → H+8	Route POST /ai/generate-pipeline. Prompt → JSON de pipeline complet avec nœuds + connexions.
H+8 → H+12	Route POST /ai/generate-transform pour le Node 09. SQL DuckDB depuis texte libre.
H+12 → H+16	Connexion frontend ↔ IA. Animation "IA en train de penser...". Injection nœuds sur canvas.
H+16 → H+20	Suggestions intelligentes de colonnes. Affinage des prompts avec vrais datasets.
H+20 → H+24	Tests intensifs. Préparer le flow de démo parfait.

P5 — DevOps · Intégration · Démo

Stack : Docker, GitHub, datasets

Horaire	Tâche
H+0 → H+2	Setup repo GitHub commun. Structure dossiers front + back. Variables d'env (.env). README.
H+2 → H+6	Proxy React ↔ FastAPI (CORS). Docker Compose. Hot reload en dev pour tout le monde.
H+6 → H+10	Préparer les datasets de démo : transactions_banque.csv (1000 lignes), clients.json (300 clients).
H+10 → H+16	Tests d'intégration end-to-end. Pipeline complet qui fonctionne. Remonter les bugs.
H+16 → H+20	Construire le script de démo parfait. Scénario bancaire réaliste en 3 minutes.
H+20 → H+24	Répétition démo x3. Slides de présentation. Vérifier que tout tourne sans internet.

Collaboration P1 & P2

Travailler ensemble sans se heurter

Séparation stricte des zones

Zone	Propriétaire	Fichiers	Interdit à l'autre
Canvas (React Flow)	P1 ONLY	src/components/canvas/**	P2 ne touche jamais
Panel gauche (nœuds)	P1 ONLY	src/components/panel/**	P2 ne touche jamais
Inspector (droite)	P2 ONLY	src/components/inspector/**	P1 ne touche jamais
Console / Preview	P2 ONLY	src/components/console/**	P1 ne touche jamais
Store (partagé)	P1 écrit nodes/edges P2 écrit nodeConfigs	src/store/pipelineStore.js	Modif structurelle = discussion obligatoire

Le contrat du store partagé

Défini à H+1, structure jamais modifiée seule :

```
const pipelineStore = {
  // P1 ECRIT – P2 LIT SEULEMENT
  nodes: [], // nœuds sur le canvas
  edges: [], // connexions entre nœuds
  selectedNode: null, // nœud cliqué → P2 affiche l'Inspector
  // P2 ECRIT – P1 LIT SEULEMENT
  nodeConfigs: {}, // config { nodeId: {params} }
  // P3 ECRIT – P1+P2 lisent
  executionResults: {} // { nodeId: {rows, status} }
}
```

Workflow Git

Branche	Qui	Règle
main	Personne directement	Merge final uniquement avant démo
dev	P5 gère les merges	Branche d'intégration commune
feat/canvas	P1 ONLY	Push toutes les 2h minimum
feat/inspector	P2 ONLY	Push toutes les 2h minimum

feat/backend	P3 ONLY	Push toutes les 2h minimum
feat/ai	P4 ONLY	Push toutes les 2h minimum

Charte Graphique — Tokens

Design System DataPipe v1.0 — Dark Industrial Precision

Philosophie visuelle

Dark industrial precision. Fond charcoal quasi-noir, accent cyan électrique, bordures à 4-10% d'opacité — ultra fines, signature du produit. Tout passe par des variables CSS --dp-*. Aucune valeur codée en dur.

Palette de couleurs

Token	Hex	Usage
--dp-bg-void	#080a0f	Fond du canvas (le plus sombre)
--dp-bg-base	#0c0e14	App shell principal
--dp-bg-surface	#10131b	Sidebar, panels
--dp-bg-elevated	#151921	Cards, nodes
--dp-bg-float	#1c2130	Dropdowns, tooltips
--dp-cyan	#00d4ff	Accent principal — bouton primary, focus, liens
--dp-green	#00e5a0	Succès, nœuds Source
--dp-amber	#ffb547	Warning, running, nœuds Output
--dp-red	#ff5370	Erreur, danger
--dp-violet	#a78bfa	IA, bonus, nœuds AI Transform
--dp-text-1	#f0f3ff	Texte primaire
--dp-text-2	#8e97b4	Texte secondaire
--dp-text-3	#525c7a	Texte muted

Typographie

Font	Usage	Poids disponibles
Plus Jakarta Sans	Toute l'interface UI — labels, boutons, titres, descriptions	300 / 400 / 500 / 600 / 700
IBM Plex Mono	Toutes les données — valeurs de table, colonnes, code SQL, métriques	300 / 400 / 500 / 600

Border Radius — Règles d'usage

Jalons sur 24h — Planning

Les checkpoints obligatoires pour livrer

Timeline critique

- | | |
|-------------|--|
| H+2 | Repo setup, stack lancée, layout global visible dans le navigateur |
| H+6 | Canvas nodal drag & drop fonctionnel — on peut glisser un nœud |
| H+10 | Pipeline CSV → Filtre → Table Preview qui s'exécute sans bug |
| H+14 | Join + Agrégation + Bar Chart fonctionnels — pipeline complet possible |
| H+16 | IA génère un pipeline depuis texte en français — feature bonus active |
| H+18 | Démo complète qui tourne de bout en bout sans intervention |
| H+21 | Répétition de la démo × 2. Slides prêtes. Pas de nouveau code. |
| H+24 | PRÉSENTATION AUX JURÉS — pipeline bancaire live en 3 minutes |

Règle d'or : le gel du code

À H+21, PLUS AUCUN NOUVEAU CODE. On polish uniquement ce qui existe. Le pire scénario d'un hackathon c'est de casser quelque chose à H+23. P5 verrouille le repo. On répète la démo.

Le Script de Démo Parfait

Ce que vous montrez aux jurés — dans l'ordre

Contexte à poser (30 secondes)

"Imaginez un contrôleur de gestion dans une banque régionale. Il reçoit chaque mois 4 fichiers CSV d'agences différentes. Il passe 2 jours dans Excel. DataPipe lui permet de faire ça en moins de 2 minutes, sans une ligne de code."

Flow de la démo — 3 minutes chrono

00:00 - 00:20	Ouvrir DataPipe. Montrer le canvas vide avec la grille. Montrer le panel gauche avec les nœuds.
00:20 - 00:45	Glisser un nœud CSV Import sur le canvas. Uploader transactions_banque.csv. Aperçu : "1 247 lignes, 8 colonnes détectées."
00:45 - 01:10	Ajouter un nœud Filtre. Connecter. Config : montant > 50 000. Appuyer Exécuter. Voir le nœud s'allumer en vert : "342 lignes retenues."
01:10 - 01:35	Ajouter un nœud Join. Uploader clients.json. Connecter. LEFT JOIN sur client_id. Exécuter : "298 matches trouvés."
01:35 - 02:00	Ajouter Agrégation GROUP BY région + SUM(montant). Exécuter. Voir les 7 régions dans la console.
02:00 - 02:15	Ajouter un nœud Bar Chart. Connecter. Axe X = région, Axe Y = total_montant. Graphique apparaît.
02:15 - 02:45	MOMENT IA : Taper dans le prompt box "Maintenant fais la même chose mais groupe par mois et ajoute la moyenne". Cliquer Générer. Les nœuds apparaissent automatiquement sur le canvas.
02:45 - 03:00	Export CSV. Télécharger. "Ce pipeline qui prenait 2 jours dans Excel prend 2 minutes dans DataPipe."

Les 3 phrases de closing

"DataPipe est open-source, tourne en local sans abonnement SaaS."

"Il s'adresse aux 80% d'analystes africains qui n'ont pas d'ingénieur data."

"Le marché ETL mondial dépasse 13 milliards de dollars. Nous ciblons sa version accessible."

Bon Hackathon à toute l'équipe DataPipe ! ■

J.U.I.N 2026 · Édition Cursor · Thème 09 · ETL Visuel pour Pipelines Bancaires